

## Concept: Pure Vs Impure

1-Pure: ពេលដែលយើងប្រើប្រាស់ Pipes មានអីផ្លាស់ប្តូរបានវា ធ្វើការ Update

it is by default in Angular

2-Impure: when click then it changes

### Pure vs Impure Pipes

When does Angular run pipe.transform()?

Custom Truncate Pipe

**TS** truncate.pipe.ts

- ✓ @Pipe decorator
- ✓ PipeTransform interface
- ✓ transform(value, ...) logic

### Pure vs Impure Pipes

Angular Change Detection

↓

transform()

↓

Angular decides when to call transform()

Pure Pipes vs Impure Pipes

### Pure vs Impure Pipes

Impure pipes run every change detection cycle

#### Angular Change Detection

Triggered by:

- Click
- Keypress
- Timer
- ✓ HTTP Response ...

#### Change Detection Cycle

Runs every cycle

#### Impure Pipe (transform())

Happens very frequently

## Pure vs Impure Pipes

Why you must be careful



If `transform()` does heavy work, an impure pipe can slow down the app.

- Impure : វាត្រូវធ្វើការ update all the time when something changes so the system will show.

Example:

>> ng g pipe truncate-impure

```
TS truncate-impure-pipe.ts X
src > app > TS truncate-impure-pipe.ts > TruncateImpurePipe > transform
1  import { Pipe, PipeTransform } from '@angular/core';
2
3  @Pipe({
4    name: 'truncateImpure',
5    pure: false
6  })
7  export class TruncateImpurePipe implements PipeTransform {
8
9    transform(value: string, limit: number = 50): string
10   {
11     console.log('Impure pipe executed');
12     if (!value)
13       return '';
14
15     if (value.length <= limit)
16       return value;
17
18     return value.substring(0, limit) + '...';
19   }
20
21 }
22
```

- Pure: false (for enable the impure)

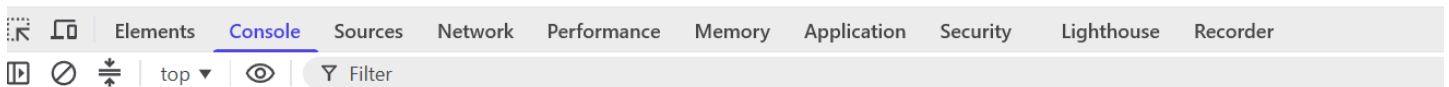
## Pure vs Impure Pipes

Counter: 0

Increment Counter

Pure Pipe: Angular is a powerful framework for build...

Impure Pipe: Angular is a powerful framework for build...



Pure pipe executed  
Pure pipe executed  
Pure pipe executed  
Pure pipe executed  
Impure pipe executed  
Impure pipe executed  
Angular is running in development mode.  
Impure pipe executed  
Impure pipe executed

```
TS truncate-impure-pipe.ts TS truncate-pipe.ts TS pipes-demo-component.ts X pipes-demo-component.ts
src > app > pipes-demo-component > TS pipes-demo-component.ts > PipesDemoComponent
1 import { CommonModule } from '@angular/common';
2 import { Component } from '@angular/core';
3 import { TruncatePipe } from '../truncate-pipe';
4 import { TruncateImpurePipe } from '../truncate-impure-pipe';
5
6 @Component({
7   selector: 'app-pipes-demo-component',
8   imports: [CommonModule, TruncatePipe, TruncateImpurePipe],
9   templateUrl: './pipes-demo-component.html',
10  styleUrls: ['./pipes-demo-component.css'],
11 })
12 export class PipesDemoComponent {
13   sampleDate: Date = new Date('2030-12-31T17:30:00+05:30');
14   message: string = 'angular pipes are POWERFUL features';
15   price: number = 1234.567;
16   completionRate: number = 0.856;
17   user: any = { name: 'John Doe', age: 30 };
18
19   description: string =
20     'Angular is a powerful framework for building modern web applications with TypeSc
21
22   counter: number = 0;
23   incrementCounter(): void {
24     this.counter = this.counter + 1;
25   }
26 }
27
```

```

<!-- Pure vs Impure Pipes -->
<section>
  <h2>Pure vs Impure Pipes</h2>

  <p>Counter: {{counter}}</p>

  <button (click)="incrementCounter()">Increment Counter</button>

  <p>Pure Pipe: {{description | truncate:40 }}</p>
  <p>Impure Pipe: {{description | truncateImpure:40 }}</p>
</section>
</div>

```

When use pure:



**Recommendation**

- ✓ In most cases, stick with **PURE** pipes
- ✓ Pure pipes are the **DEFAULT**
- ✓ More **performant**
- ✓ Works **correctly** for most use cases

When use impure:

When would you consider an impure pipe?

**Array is modified**  
(same reference)

**items[]**  
 Array reference: **A**

↓  
push() / remove()

**items[]**  
 Array reference: **A**

**Pure Pipe does NOT run**

**Pure Pipe (default)**

- Input reference unchanged (**A - A**)
- Angular treats it as **NOT** a pure change
- Transform() **NOT executed**

**Displayed value may not update**

**items[]**  
 Array reference: **A**

## Prefer immutable updates

Create new array  
(new reference)

Before

items[]

Array reference: A

create new array

items = [...items, newItem]

After

items[]

Array reference: B

Pure Pipe runs



Pure Pipe

Reference changed (A - B)

Angular detects a **pure change**

Transform() **executed**

Keep pipes pure unless you truly need impure.

items[]

Array reference: B

## Quick Recap - Pure Pipes



**Pure Pipe** (Default in Angular)

- Default behavior in Angular
- Runs only when input reference changes
- Efficient: avoids recalculating same result

Input Reference



Pipe.transform()

Runs only on reference change

**Pure pipes** are the **right choice** for most applications.

## Quick Recap - Impure Pipes & Recommendation



### Pure Pipe

- Runs only on reference change

Efficient



### Impure Pipe

- Runs on every change detection cycle
- Make it impure: `pure: false` in `@Pipe`

Key setting

```
@Pipe({ name: 'x', pure: false })
```

- Detects changes inside arrays/objects

Use only when truly needed

Use impure pipes only when truly needed -always be mindful of performance.